

SKILL - WEEK 7

Task -1 :

To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    char input[200];

    printf("Enter command with append redirection: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    char *redirect = strstr(input, ">>");

    if (redirect == NULL)
    {
        printf("No append redirection found.\n");
        return 1;
    }

    *redirect = '\0';

    char *command = input;
    char *filename = redirect + 2;

    while (*filename == ' ')
        filename++;

    while (*command == ' ')
        command++;

    int fd = open(filename, O_WRONLY | O_CREAT | O_APPEND, 0644);

    if (fd == -1)
    {
        perror("File open");
        return 1;
    }

    int saved_stdout = dup(STDOUT_FILENO);

    if (fork() == 0)
    {
        dup2(fd, STDOUT_FILENO);
        close(fd);

        char *args[20];
        int i = 0;
        char *token = strtok(command, " ");

        while (token != NULL && i < 19)
        {
            args[i++] = token;
            token = strtok(NULL, " ");
        }

        args[i] = NULL;

        execvp(args[0], args);

        perror("exec");
        exit(1);
    }

    wait(NULL);

    dup2(saved_stdout, STDOUT_FILENO);
    close(saved_stdout);
    close(fd);

    printf("Append redirection completed successfully.\n");

    return 0;
}
```

OUTPUT:

```
Radhakrishna@04:~# nano append_redirect.c
Radhakrishna@04:~# gcc append_redirect.c -o append_redirect
Radhakrishna@04:~# ./append_redirect
Enter command with stderr redirection: echo First line >> output.txt
Append redirection completed successfully.
Radhakrishna@04:~# cat output.txt
First line
Radhakrishna@04:~# ./append_redirect
Enter command with stderr redirection: echo second line >> output.txt
Append redirection completed successfully.
Radhakrishna@04:~# cat output.txt
First line
second line
Radhakrishna@04:~#
```

Task - 2:

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    char input[200];

    printf("Enter command with stderr redirection: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    char *redirect = strstr(input, ">>");

    if (redirect == NULL)
    {
        printf("No stderr redirection found.\n");
        return 1;
    }

    *redirect = '\0';

    char *command = input;
    char *filename = redirect + 2;

    while (*filename == ' ')
        filename++;

    while (*command == ' ')
        command++;

    int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd == -1)
    {
        perror("File open");
        return 1;
    }

    int saved_stderr = dup(STDERR_FILENO);

    if (fork() == 0)
    {
        dup2(fd, STDERR_FILENO);
        close(fd);

        char *args[20];
        int i = 0;
        char *token = strtok(command, " ");

        while (token != NULL && i < 19)
        {
            args[i++] = token;
            token = strtok(NULL, " ");
        }

        args[i] = NULL;

        execvp(args[0], args);

        perror("exec");
        exit(1);
    }

    wait(NULL);

    dup2(saved_stderr, STDERR_FILENO);
    close(saved_stderr);
    close(fd);

    printf("stderr redirection completed successfully.\n");

    return 0;
}
```

OUTPUT:

```
Radhakrishna@04:~# nano stderr_redirect.c
Radhakrishna@04:~# gcc stderr_redirect.c -o stderr_redirect
Radhakrishna@04:~# ./stderr_redirect
Enter command with stderr redirection: ls nonexistent 2> error.txt
stderr redirection completed successfully.
Radhakrishna@04:~# cat error.txt
ls: cannot access 'nonexistent': No such file or directory
Radhakrishna@04:~#
```